Tensor Parallelism with CUDA - Multi-GPU Matrix Multiplication

Tensor Parallelism from Scratch

DENNIS KENNETZ

MAR 08, 2025

Intro:

In the last post, I broke down a matrix multiplication on paper (in the article) before introducing the first bit of CUDA code! I concluded that article by performing a matrix multiplication on a single GPU, and showed why matrix multiplications were such a great target for multi-GPU calculations. If you missed it, check it out here.

For those just jumping into the series, I came to machine learning from a bioinformatics algorithms background and don't have a ton of experience with the mechanics underlying the tools I use every day. However, I do understand how GPUs work, and how to write CUDA code to utilize the GPUs to do work, as I spent 99% of my dev work in my previous role writing HPC algorithms in CUDA / C++. Now, a lot of this is abstracted away from me by some amazing Python libraries like PyTorch, TensorFlow, and JAX so I still don't really get to interact with the "meat" of what runs these amazing ML algorithms - and that is what I'm setting out to explore.

In this article, we will spend much more time looking at CUDA code, as the operations of the matrix multiplication are the same from last week, they will just be

split across two GPUs this time! This does require some playing with offsets to transfer correct data. They will also be performed on larger matrices, so we can investigate the impact of multiple GPUs on performance. To do this, we will look at additional CUDA API calls such as cudaSetDevice, cudaStream, cudaMemcpyAsync, and cudaEvent in conjunction with our matmul kernel.

All code samples can be found in the GitHub:

https://github.com/drkennetz/cuda_examples/blob/main/TensorParallelFromScratch/02_matmul_tp/matmul_tp_big.cu

Let's start at the top

The main routine of any program tells the whole story. If you walk through it, you can exactly follow the execution of the program - so that's where we'll start! After initializing memory (this is important, but not the point of this article - we'll expand on the different types of memory allocations in a separate article), we see our first piece of relevant code to the exercise of the multi-GPU matmul:

```
#define M 4096  // Rows of A, Rows of C
#define N 8192  // Columns of B, Columns of C
#define K 1024  // Columns of A, Rows of B
#define NGPUS 2 // Number of GPUs to use for computation

int main() {
    ...
    cudaCheckError(::cudaMallocHost(&A, M * K * sizeof(int)));
    cudaCheckError(::cudaMallocHost(&B, K * N * sizeof(int)));
```

```
    cudaCheckError(::cudaMallocHost(&C, M * N * sizeof(int)));
    ...
```

If we look at M, N, K at the top we can see the following dimensions for each matrix:

- Input matrix A: [4096, 1024]
- Input matrix B: [1024, 8192]
- Resultant matrix C: [4096, 8192]

If we recall from the last article, the “like” dimension (1024 in this case) is how we are able to multiply our matrices, resulting in an output matrix with the two “unlike” dimensions (4096, 8192). In this case, we have matrix A with ~4M integers being multiplied by matrix B with ~8M integers, resulting in a matrix C with 32M integers. This is not huge, but also not small either - we should be able to see a speedup by splitting calculations… But how do we do that split?

The first important piece of information regarding the split can be found in this like:

```
    // Calculate split for tensor parallelism across N dimension
    const int cols_per_gpu = N / NGPUS;
```

If you recall, in the matrix multiplication we have to multiply every row by every column to get the final result. In this line of code, we are just defining how many columns should be calculated on each GPU. A consideration to be made here is when the number of GPUs don’t evenly divide into the number of columns, in which case

you should round up and just perform a few early exits in the kernel (which is why this check is often included):

```
if (row < m && local_col < col_size) {
```

Events and Streams

For all my micro-services folks out there, cudaStreams and cudaEvents are actually very similar to events and streams in something like Kafka. Let's take a look:

```
    // Create streams and events per GPU
    cudaStream_t streams[NGPUS];
    cudaEvent_t startEvents[NGPUS], stopEvents[NGPUS];

    for (int gpu = 0; gpu < NGPUS; gpu++) {
        cudaCheckError(::cudaSetDevice(gpu));
        cudaCheckError(::cudaStreamCreate(&streams[gpu]));
        cudaCheckError(::cudaEventCreate(&startEvents[gpu]));
        cudaCheckError(::cudaEventCreate(&stopEvents[gpu]));
    }
```

Streams

Let's touch on two types of parallelism, data and task:

- Data Parallelism: splits the same computation across multiple threads / GPUs, processing different parts of data simultaneously (like this whole tensor-

parallelism series describes)

- Task Parallelism: Allows overlapping execution of different operations (like memory transfers or computations) to maximize GPU utilization.

So, while GPUs are inherently data parallel, they are not necessarily task parallel, which is where cudaStreams come in!

A CUDA stream is basically just a sequence of GPU operations that execute in order within a stream, but can run concurrently with operations in other streams. In fact, even when a stream isn't specified or created, streams are used in CUDA.

By default, CUDA operations execute in Stream 0 which is synchronous and blocks the CPU. Like the example above, we can create custom streams which allow overlapping computations and memory transfers.

A good mental model for task parallelism is two roommates shopping in a grocery store. They drive in the car together and arrive at the store at the same time. When they get to the store, they both split and do their individual shopping. They can shop as fast or slow as they want and checkout when they want. But to leave, the friend who finishes first always must wait for the second friend before they can drive home (if they are nice). This can have benefits - like while friend one is waiting, they could scroll on the gram, make a todo list for next week, or read this article (IE they can use that time for something else, just like compute could be used for something else). However, the program cannot complete until both friends are done (or all tasks complete).

More on streams in a second, but first we will touch on cudaEvents:

Events

Events in CUDA are lightweight synchronization primitives primarily used for:

1. Measuring execution time of GPU operations
2. Synchronizing streams to ensure proper execution order
3. Signaling events when specific tasks complete

As my examples are fairly simple, only (1) and (2) are seen in this code. I am using them to synchronize streams and measure timing.

Back to Streams

```
    // Set up each GPU
    for (int gpu = 0; gpu < NGPUS; gpu++) {
        cudaCheckError(::cudaSetDevice(gpu));

        // Allocate memory on current GPU
        cudaCheckError(::cudaMalloc(&d_A[gpu], M * K * sizeof(int)));
        cudaCheckError(::cudaMalloc(&d_B[gpu], K * N * sizeof(int)));
        cudaCheckError(::cudaMalloc(&d_C[gpu], M * cols_per_gpu * sizeof(int)));

        // Copy input matrices to current GPU
        cudaCheckError(::cudaMemcpyAsync(d_A[gpu], A, M * K * sizeof(int),
cudaMemcpyHostToDevice, streams[gpu]));
        cudaCheckError(::cudaMemcpyAsync(d_B[gpu], B, K * N * sizeof(int),
cudaMemcpyHostToDevice, streams[gpu]));
```

As mentioned, we can create **custom** streams which require sequential execution within a stream, but are asynchronous compared to other streams. In this block here,

we are:

1. Allocating matrix memory on each GPU. For simplicity, I'm just throwing the whole input matrices up, but we could partition them further to reduce memory (I think)). However, for the output matrix, I'm only allocating the fraction of the output memory per each GPU, because I'll only be calculating on a fraction of the columns (see d_C memory allocation)
2. I'm performing stream based asynchronous copies. This means I'm launching a memcpy on the host, but it can execute on its stream in the background and does not block this same copy from happening on other streams. All the copies within the stream are synchronous, but they are asynchronous with respect to other streams. In this case, I created two streams. I am performing asynchronous copies to one GPU on one stream, and asynchronous copies to the other GPU on the other stream.

The matmul setup

```
        // Set grid and block dimensions for this GPU's portion
        const dim3 threadsPerBlock(16, 16);  // Using 16x16 thread blocks for better
occupancy
        const dim3 numBlocks(
            (cols_per_gpu + threadsPerBlock.x - 1) / threadsPerBlock.x,
            (M + threadsPerBlock.y - 1) / threadsPerBlock.y
        );

        const int col_start = gpu * cols_per_gpu;

        cudaCheckError(::cudaEventRecord(startEvents[gpu], streams[gpu]));
```

```
        matMulKernelTP<<<numBlocks, threadsPerBlock, 0, streams[gpu]>>>(
            d_A[gpu], d_B[gpu], d_C[gpu],
            M, N, K, col_start, cols_per_gpu
        );

        // Record stop event for this GPU
        cudaCheckError(::cudaEventRecord(stopEvents[gpu], streams[gpu]));
    }
```

Warning: This part may be hard to follow, and that's okay. The important takeaway here is that:

- We have to tell CUDA how we want to launch work (fine-grained control)
- We do a bit of math to make sure we launch enough work to perform all the calculations in our matmul

How CUDA launches work is incredibly important to get right for performance, especially at critical parts like big calculations. I'm not going to spend much time on this as it should (and likely will) be an entire series on its own.

Back to the story: Here, we get into the launching of our matrix multiplication. This part is a bit math and a bit tuning, and I'm probably not optimized here (feel free to fix this for me). I'm focusing on two important dimensions here with the CUDA `dim3` data type (x, y):

threadsPerBlock: CUDA typically organizes threads in 2D thread blocks, commonly using 16 x 16 threads per block as a starting point.

- This is because 16 x 16 = 256 threads per block is a reasonable fit for most modern GPUs
- Each thread computes an element of output matrix C
- This size aligns well with shared memory tiling operations (which aren't being used here, but should be used in general matrix multiplications - maybe next time)

numBlocks: Each thread block computes a sub matrix (tile) of C, so the number of blocks is determined by the number of columns calculated on each GPU, divided by the number of threads per block. This should align fairly well with reason:

- We have "cols_per_gpu" calculations to do in each block, divided by the number of threads in that block
- We have "M" (rows) calculations to do in each block, divided by the number of of threads in that block

It may seem weird that columns is in the x index (or not) and rows is in the y index (or not), but this aligns with the CUDA thread indexing convention:

- threadIdx.x → column index
- threadIdx.y → row index

The reason for the convention is that these indexing patterns align with memory layouts in row-major order where `cols_per_gpu` maps to `blockIdx.x` for horizontal traversal, and rows `M` map to `blockIdx.y` for vertical traversal (touched on these in the last article).

The reason for the additional buffer of `cols_per_gpu + threadsPerBlock.x - 1` is to ensure correct rounding up so that even if cols_per_gpu or M is not perfectly divisible by 16, we still launch enough blocks.

The Kernel launch:

```
        const int col_start = gpu * cols_per_gpu;

        cudaCheckError(::cudaEventRecord(startEvents[gpu], streams[gpu]));

        matMulKernelTP<<<numBlocks, threadsPerBlock, 0, streams[gpu]>>>(
            d_A[gpu], d_B[gpu], d_C[gpu],
            M, N, K, col_start, cols_per_gpu
        );

        // Record stop event for this GPU
        cudaCheckError(::cudaEventRecord(stopEvents[gpu], streams[gpu]));
```

The col_start line here is interesting. We actually tell our kernel where we want it to start work because we are only doing half the calculations on each GPU. Some additional information about the launch is that we are launching it on our custom stream, so the calculation can happen asynchronously on each GPU. This is not strictly necessary for work happening on separate GPUs, but using streams is good practice because it does guarantee the overlapping of kernel execution with memory transfers, and multiple kernel launches on the same GPU. It also allows us to capture event times independently on each GPU for the kernel.

The Kernel:

```
// Kernel to perform matrix multiplication on a portion of the matrices
__global__ void matMulKernelTP(int *A, int *B, int *C, int m, int n, int k, int
col_start, int col_size) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int local_col = blockIdx.x * blockDim.x + threadIdx.x;
    int col = col_start + local_col;

    if (row < m && local_col < col_size) {
        int value = 0;
        for (int i = 0; i < k; i++) {
            value += A[row * k + i] * B[i * n + col];
        }
        C[row * col_size + local_col] = value;
    }
}
```

The main point of separation here from the matmul kernel in our previous article is that we introduce the `local_col`. If you recall, we are only calculating half the columns on each GPU - so local col would only go from 0-4095 because our total column is 8192. We provide a `col_start` offset to tell our kernel to index into our input matrix B at the offset for these calculations. That means:

- GPU 0 will calculate all rows for columns 0-4095
- GPU 1 will calculate all rows for columns 4096-8192

One additional “strange” thing for CPU programmers is that all “arrays” in CUDA are flat. IE, we do not have an actual 2D construct here. Instead we have to “pretend” by

doing the math to calculate offsets. It's not so bad once you get used to it:

- Output index for row 0 column 0 is: 0
- Output index for row 1 column 0 is: 8192
- Output index for row 2 column 0 is: 16384

Because each row has 8192 columns, the next "row" start (in the flattened output array) is offset by the number of columns. Considering the following 2D list in python:

```
x = [
  [1, 2],
  [3, 4]
]
x[0][0] = 1
x[1][0] = 3
```

In CUDA, this would be flattened to:

```
x = [1, 2, 3, 4]
```

So you have to use your dimensions to calculate offsets.

Let's Check the Perf

OK - dense article, I get it. But was all this extra fluff worth it? Let's see. I have timers set, and I can check results just by flipping NGPUS from 1 to 2.

1 GPU result:

```
GPU 0 execution time: 37.0033ms
```

2 GPU result:

```
GPU 0 execution time: 18.238ms
GPU 1 execution time: 18.3068ms
```

Perhaps as expected, the total execution time scales linearly as we add more GPUs (because we're doing proportionally less calculations). This is why matmul is such a great operation to parallelize across GPUs! We can perform exact parts of the calculation on each GPU and combine them later for the whole result.

To confirm the correctness of the result, I also run a naive matmul on the CPU, which takes ~100s for a matrix of the same size which is roughly 50000x longer than it took on the GPU. You could say, "yeah but the CPU matmul isn't optimized!" And my counter would be, "yeah, neither is the GPU matmul!" Matrix multiplications are some of the more widely studied calculations in this space, and I am more interested in studying mechanics. If you are interested in an article describing how someone might think about deep optimizations to matrix multiplications, this article is awesome:

https://salykova.github.io/sgemm-gpu

Where the author beats cuBLAS (NVIDIAs Linear Algebra library) at matrix multiplication.

Final Remarks

This article was dense - I know that. But to be honest, we've moved to that part of the story. Tensor parallelism is a beast, and it will continue to get more and more technical as each article builds upon the previous. In the next article, I'll discuss transformers MLP layers, Attention Layers and expand on how tensor parallelism is applied in those contexts. With each article, we'll first spend some time understanding what they are before diving into coding.

Thanks for reading, and if you'd like to follow the code, you can do so at the GitHub repo:

https://github.com/drkennetz/cuda_examples/tree/main

I hope you found this useful!

17 Likes

Discussion about this post

Comments Restacks